

CS 112

Introduction to Data Structures

WEEK 06

Linked Lists

Eric Araújo

Calvin University · Fall 2026

This Week

- 1 The problem with dynamic arrays
- 2 A clever solution, and how far to take it
- 3 `Node` and `List`
- 4 Two classes, one exposed
- 5 `prepend` , and walking the list
- 6 The destructor **KEY**
- 7 Dynamic array vs. linked list

Which Operations Are Expensive?

- `append()` when the array is full
- `insert()` in the middle, especially at the front
- `delete()` in the middle, especially at the front

A Clever Solution?

Store the array in smaller chunks, a collection of smaller arrays.

Advantages

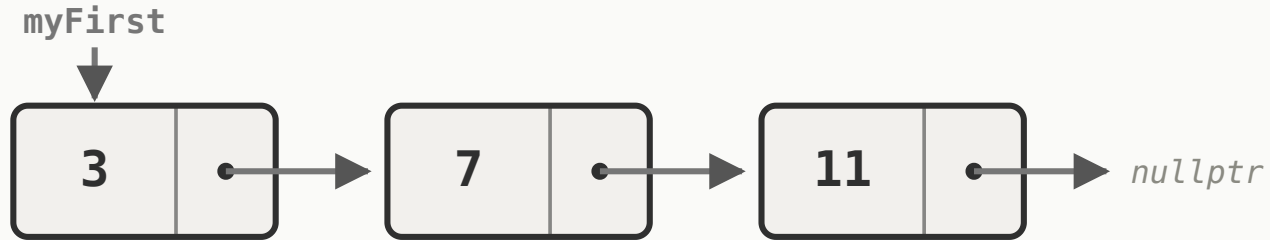
- Insert and delete shift fewer items

Disadvantages

How Big Is a Chunk?

- 5 items each? 3? 2?

A Linked List



Each item lives in its own **Node**, which also holds the address of the next one. The nodes form a chain.

Two Classes

Node: one link

```
class Node {  
    public:  
        Item  myItem;  
        Node *myNext;  
};
```

List: the container

```
class List {  
    private:  
        int    mySize;  
        Node *myFirst;  
        Node *myLast;  
};
```

Two Classes, One Exposed

```
class List {  
    public:  
        getSize(), isEmpty(), append(), prepend() ...  
  
    private:  
        class Node {                                // nobody outside List can even name this  
            public:  
                Item  myItem;  
                Node *myNext;  
        };  
  
        Node *myFirst, *myLast;  
};
```

Users of a container do not need to know its inner workings.

TALK TO YOUR NEIGHBOR · TTYN

Which line fails to compile?

A. `List<int> L;`

B. `List<int>::Node *n;`

C. `L.append(4);`

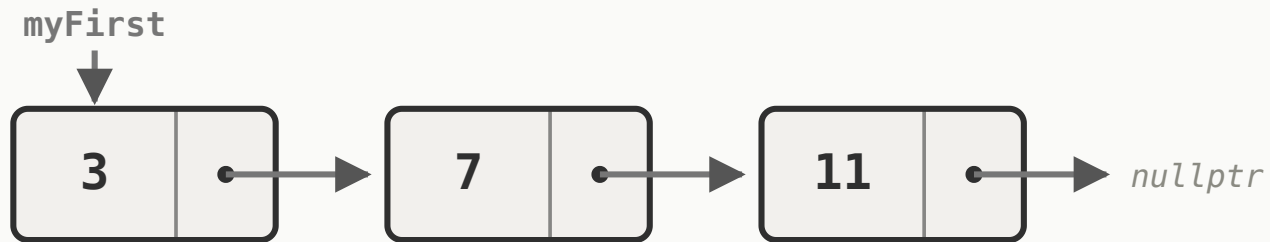
D. `cout << L.getSize();`

TALK TO YOUR NEIGHBOR · TTYN

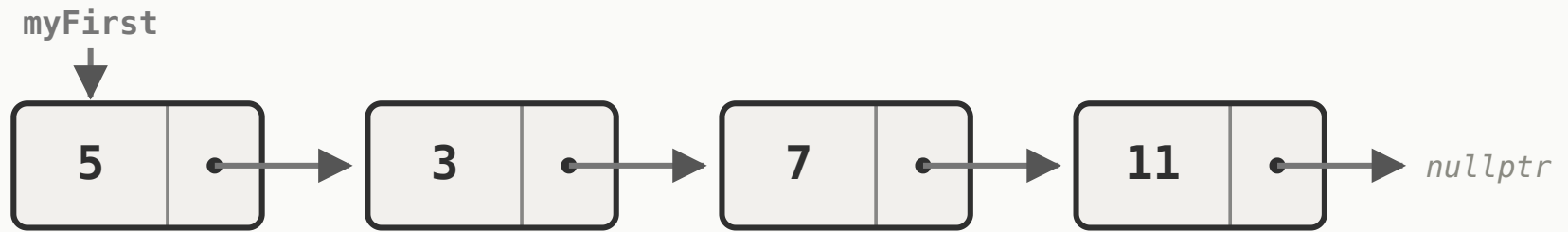
Outline the steps to implement `prepend(item)` .

Does your algorithm handle the case where the list is **empty**?

prepend(5)



prepend(5)



```
void List::prepend(Item it) {  
    myFirst = new Node(it, myFirst);    // point the new one at the old first  
    if (mySize == 0) { myLast = myFirst; }    // empty list: it is last too  
    ++mySize;  
}
```

Write the Code For...

- `List` and `Node` constructors
- `getFirst`, `getLast`, `getSize`
- `prepend`, `writeTo(ostream &)`
- `List` and `Node` destructors

Destructor

Two things happen, **in this order**:

1. The code for the destructor runs
2. The memory for the object goes back

Called when you delete an object, and when an object goes out of scope.

The Chain Reaction

```
Node::~~Node() {  
    delete myNext;           // destroy the rest of the list  
}  
  
List::~~List() {  
    delete myFirst;         // starts the chain  
}
```

The order is why this works. `~Node` runs *before* the node's memory goes back, so `myNext` is still readable. `delete nullptr` is harmless, and that is what stops the chain.

TALK TO YOUR NEIGHBOR · TTYN

A `List` holding 4 items goes out of scope. How many `Node` destructors run?

- A.** 1, only the first node is deleted
- B.** 4, each node deletes the next
- C.** 0, the compiler handles it
- D.** 5, including the nullptr at the end

Dynamic Array vs. Linked List

Operation	Dynamic array	Linked list
<code>append()</code>	amortized constant	constant
<code>prepend()</code>	linear	constant
insert / remove in the middle	linear	linear
delete the first item	linear	constant
delete the last item	constant	linear (!)
traverse, e.g. print	linear	linear
find the n-th item	constant	linear
space	minimal	a pointer per item

TALK TO YOUR NEIGHBOR · TTYN

Deleting the last node of a linked list is...

- A.** constant, `myLast` points right at it
- B.** linear, you walk the list to find the one before it
- C.** amortized constant
- D.** constant, the destructor chain handles it

TALK TO YOUR NEIGHBOR · TTYN

Thousands of values, always added at the front, count unknown. Which?

- A.** An array
- B.** A dynamic array
- C.** A linked list
- D.** Any of them, it makes no difference

TALK TO YOUR NEIGHBOR · TTYN

Same, but always added at the end, then read in random order.

- A.** An array
- B.** A dynamic array
- C.** A linked list
- D.** Either B or C, appending is cheap on both

Week 06 Recap

- A linked list trades **contiguity** for **flexibility**
- Each `Node` holds an item and the address of the next
- `Node` is private to `List`: users never see the chain
- `prepend` is constant; finding the n-th item is linear
- The empty list is where the bugs hide
- Destructors chain, and `delete nullptr` ends the chain safely

Next: a08 →

This deck stands on earlier CS112 materials by
Joel Adams and **Victor Norman** · adapted and extended by **Eric Araújo**

